

Vou organizar tudo em seções bem definidas. Começando pelo diagrama de arquitetura geral do sistema, depois respondo cada tópico em detalhe. Boa visão do sistema. Agora vou detalhar cada ponto que vocês levantaram.

---

## 1. Modelagem do banco de dados

### Tabela `usuarios` — melhorias e campos sugeridos

Além do que vocês já definiram (nome, CPF, código, e-mail, senha), vale adicionar:

```
CREATE TABLE usuarios (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  codigo      VARCHAR(12) UNIQUE NOT NULL,      -- gerado automaticamente  
  cpf         VARCHAR(11) UNIQUE NOT NULL,      -- somente dígitos, sem máscara  
  nome_completo VARCHAR(150) NOT NULL,  
  email       VARCHAR(255) UNIQUE NOT NULL,  
  senha_hash  VARCHAR(255) NOT NULL,           -- bcrypt  
  telefone    VARCHAR(20),                     -- para integração WhatsApp/Telegram  
  telegram_chat_id BIGINT,                     -- bind do bot Telegram  
  whatsapp_id VARCHAR(100),                    -- bind do número no WhatsApp  
  saldo_inicial NUMERIC(15,2) NOT NULL DEFAULT 0,  
  moeda        CHAR(3) NOT NULL DEFAULT 'BRL',  
  ativo        BOOLEAN NOT NULL DEFAULT TRUE,  
  email_verificado BOOLEAN NOT NULL DEFAULT FALSE,  
  token_verificacao VARCHAR(100),              -- para confirmar e-mail  
  ultimo_login  TIMESTAMPTZ,  
  tentativas_login SMALLINT DEFAULT 0,         -- controle de brute force  
  bloqueado_ate TIMESTAMPTZ,                   -- bloqueio temporário após X tentativa  
  refresh_token_hash VARCHAR(255),             -- para renovação do JWT  
  criado_em     TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  atualizado_em TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
-- Chave composta para lookup seguro  
CREATE UNIQUE INDEX idx_usuario_cpf_codigo ON usuarios (cpf, codigo);
```

O campo `saldo_inicial` é importante porque o PMV prevê que o usuário registra o saldo atual para começar o controle — esse valor vai ser a âncora dos cálculos de saldo corrente.

---

## Tabela `transacoes` — modelagem completa

Esta é a tabela mais crítica do sistema. Precisa cobrir receitas, gastos manuais e gastos importados via PDF:

```
CREATE TABLE transacoes (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  usuario_id  UUID NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,  
  extrato_id  UUID REFERENCES extratos(id),      -- null se lançamento manual  
  tipo        VARCHAR(10) NOT NULL CHECK (tipo IN ('receita', 'gasto')),  
  descricao_original TEXT,                      -- texto bruto do extrato  
  descricao_usuario VARCHAR(255),              -- descrição digitada manualmente  
  valor       NUMERIC(15,2) NOT NULL CHECK (valor > 0),  
  data_transacao DATE NOT NULL,  
  data_lancamento TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  categoria_id  UUID REFERENCES categorias(id),  
  subcategoria  VARCHAR(100),  
  estabelecimento VARCHAR(255),  
  origem        VARCHAR(20) NOT NULL DEFAULT 'manual'  
                CHECK (origem IN ('manual', 'pdf', 'api')),  
  status_revisao VARCHAR(20) NOT NULL DEFAULT 'ok'  
                CHECK (status_revisao IN ('pendente', 'ok', 'ignorado')),  
  confianca_ia  SMALLINT CHECK (confianca_ia BETWEEN 0 AND 100),  
  recorrente    BOOLEAN DEFAULT FALSE,  
  periodo_recorrencia VARCHAR(20), -- 'mensal', 'semanal', etc.  
  observacao    TEXT,  
  criado_em     TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  atualizado_em TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX idx_transacao_usuario_data ON transacoes (usuario_id, data_transacao);  
CREATE INDEX idx_transacao_categoria ON transacoes (categoria_id);  
CREATE INDEX idx_transacao_status ON transacoes (usuario_id, status_revisao)  
  WHERE status_revisao = 'pendente';
```

O campo `confianca_ia` (0–100) é fundamental para o fluxo de revisão: abaixo de um threshold configurável (ex: 70), a transação cai como `pendente` para o usuário revisar.

---

## Tabelas de suporte necessárias

```
-- Categorias com suporte a hierarquia pai/filho
CREATE TABLE categorias (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  nome    VARCHAR(100) NOT NULL,
  categoria_pai_id UUID REFERENCES categorias(id),
  tipo    VARCHAR(10) CHECK (tipo IN ('receita','gasto','ambos')),
  icone   VARCHAR(50),
  cor_hex CHAR(7),
  padrao  BOOLEAN DEFAULT FALSE, -- categorias padrão do sistema
  usuario_id UUID REFERENCES usuarios(id) -- null = global, preenchido = personalizada
);

-- Regras de aprendizado da IA por usuário
CREATE TABLE regras_classificacao (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  usuario_id  UUID NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,
  termo      VARCHAR(255) NOT NULL, -- trecho da descrição que dispara a regra
  categoria_id  UUID NOT NULL REFERENCES categorias(id),
  subcategoria VARCHAR(100),
  confianca    SMALLINT DEFAULT 100,
  vezes_aplicada INT DEFAULT 0,
  criado_em    TIMESTAMPTZ DEFAULT NOW()
);
CREATE UNIQUE INDEX idx_regra_usuario_termo ON regras_classificacao (usuario_id, termo);

-- Extratos importados (agrupador de transações)
CREATE TABLE extratos (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  usuario_id  UUID NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,
  arquivo_nome  VARCHAR(255),
  banco_detectado VARCHAR(100),
  periodo_inicio DATE,
  periodo_fim    DATE,
  status         VARCHAR(20) DEFAULT 'processando'
```

```
        CHECK (status IN ('processando','pendente_revisao','concluido','erro')),
        total_lançamentos INT,
        hash_arquivo  VARCHAR(64), -- SHA-256 para evitar upload duplicado
        criado_em      TIMESTAMPTZ DEFAULT NOW()
    );
```

## 2. Tecnologias complementares sugeridas

Além da stack que vocês já definiram, estas adições se encaixam bem:

| Necessidade                     | Tecnologia                                      | Justificativa                                                             |
|---------------------------------|-------------------------------------------------|---------------------------------------------------------------------------|
| Cache e sessões<br>JWT          | Redis                                           | Invalida tokens antes do vencimento, armazena blacklist de refresh tokens |
| Fila de<br>processamento<br>PDF | RabbitMQ ou Amazon<br>SQS                       | Processar PDF é pesado — desacoplar da requisição HTTP evita timeout      |
| Extração de texto<br>de PDF     | Apache PDFBox (Java) ou<br>pdfplumber (Python)  | PDFBox integra direto no Spring; Python via microserviço se necessário    |
| Testes de<br>integração (Java)  | JUnit 5 + Mockito +<br>Testcontainers           | Testcontainers sobe um PostgreSQL real nos testes, sem mock frágil        |
| Documentação da<br>API          | SpringDoc OpenAPI<br>(Swagger UI)               | Geração automática a partir das anotações do Spring                       |
| Monitoramento                   | Spring Actuator +<br>Prometheus + Grafana       | Gratuitos, simples de configurar com Docker Compose                       |
| Migrations de<br>banco          | Flyway                                          | Integra nativamente com Spring Boot, controla versão do schema            |
| Relatório PDF                   | JasperReports ou iText<br>(Java)                | Geração de PDF dos relatórios financeiros para download                   |
| Variáveis de<br>ambiente        | Spring Config + <code>.env</code> via<br>Docker | Nunca credenciais no código                                               |

Para o N8N especificamente: ele vai atuar como middleware entre o WhatsApp/Telegram e o endpoint da sua API. O fluxo seria o N8N receber o PDF do bot, fazer o download e repassar via `multipart/form-data` para o endpoint `/api/extratos/upload`.

---

### 3. Regras de negócio detalhadas

#### Usuário e acesso

**RN-01 — Unicidade do CPF:** Não pode existir dois usuários com o mesmo CPF. A tentativa de cadastro com CPF já existente retorna erro `409 Conflict` com mensagem clara, sem revelar se o e-mail associado existe (prevenção de enumeração).

**RN-02 — Código do usuário:** Gerado automaticamente pelo sistema no cadastro, nunca editável pelo usuário. Formato sugerido: `FIN-XXXXXX` onde X é alfanumérico aleatório. A chave composta `cpf + código` serve como identificador externo seguro para integrações.

**RN-03 — Bloqueio por tentativas:** Após 5 tentativas de login com senha errada, a conta é bloqueada por 15 minutos. O campo `bloqueado_ate` registra o horário de desbloqueio. Após o bloqueio, o contador `tentativas_login` é zerado.

**RN-04 — Saldo inicial obrigatório:** O usuário deve registrar o saldo atual antes de iniciar qualquer lançamento. Enquanto `saldo_inicial` não for definido, o dashboard exibe um estado de “configuração pendente” e bloqueia o acesso aos relatórios.

#### Transações

**RN-05 — Integridade do valor:** Nenhuma transação pode ter `valor <= 0`. O tipo (receita ou gasto) é definido pelo campo `tipo`, não pelo sinal do valor.

**RN-06 — Data da transação vs. data de lançamento:** `data_transacao` é a data em que o gasto/receita ocorreu de fato (pode ser editada pelo usuário). `data_lancamento` é o timestamp de quando foi registrado no sistema e nunca é editável.

**RN-07 — Saldo calculado:** O saldo corrente nunca é armazenado diretamente para evitar inconsistências. É sempre calculado como: `saldo_inicial + SUM(receitas) - SUM(gastos)`. Uma view ou função no banco pode materializar isso eficientemente.

**RN-08 — Período máximo de relatório personalizado:** Relatórios customizados são limitados a 6 meses. Acima disso, o sistema retorna erro de validação com sugestão de usar os relatórios anuais pré-gerados.

**RN-09 — Edição de transação importada:** Transações vindas de PDF podem ter descrição e categoria editadas pelo usuário, mas o valor e a data original nunca podem ser alterados após confirmação (integridade do extrato).

## Processamento de PDF e IA

**RN-10 — Deduplicação de extrato:** Antes de processar um PDF, o sistema calcula o SHA-256 do arquivo e verifica se já existe um extrato com o mesmo hash para aquele usuário. Se sim, rejeita com `409` e informa que o extrato já foi importado.

**RN-11 — Fila de revisão:** Toda transação extraída de PDF entra com `status_revisao = 'pendente'` se `confianca_ia < 70`, ou `'ok'` se a IA classificou com alta confiança. O usuário pode ir direto para os relatórios com as transações de alta confiança, mas é alertado sobre as pendentes.

**RN-12 — Aprendizado com correção:** Quando o usuário corrige uma categoria na tela de revisão, o sistema salva a regra em `regras_classificacao`. Na próxima importação, antes de chamar a IA, o sistema consulta as regras do usuário — se houver match exato ou fuzzy no `termo`, aplica a regra com `confianca = 100` sem acionar a IA, economizando custo de API.

**RN-13 — Transação completamente nova:** Mesmo com regras salvas, se a descrição não tiver match com nenhuma regra e a IA retornar `confianca < 50`, a transação fica obrigatoriamente `'pendente'` e não pode ser confirmada sem categorização manual.

**RN-14 — Limite de arquivo:** PDFs aceitos devem ter no máximo 10 MB e no máximo 500 transações por arquivo. Acima disso, o sistema orienta o usuário a dividir o período.

## Categorias

**RN-15 — Categorias padrão imutáveis:** As categorias globais do sistema (marcadas com `padrao = TRUE`) não podem ser editadas ou excluídas por nenhum usuário, incluindo administradores. O usuário pode criar categorias personalizadas próprias.

**RN-16 — Exclusão de categoria com transações:** Não é permitido excluir uma categoria que possui transações vinculadas. O sistema deve sugerir migrar as transações para outra categoria antes da exclusão.

---

## 4. Segurança com JWT

A estratégia recomendada é trabalhar com dois tokens, não apenas um:

**Access Token** — vida curta (15 a 30 minutos), enviado no header `Authorization: Bearer <token>`, contém: `usuario_id`, `codigo`, `roles`, `iat`, `exp`. Nunca armazenar no `localStorage` do browser — usar cookie `httpOnly` ou memória da aplicação React.

**Refresh Token** — vida longa (7 a 30 dias), armazenado em cookie `httpOnly; Secure; SameSite=Strict`. Quando o access token expirar, o frontend faz `POST /auth/refresh` e recebe um novo par de tokens. O hash do refresh token atual é guardado em Redis com TTL correspondente — ao fazer refresh, o token antigo é invalidado (rotação de tokens).

Outras práticas importantes para a implementação Spring:

```
// Claims recomendados no payload do JWT
{
  "sub": "uuid-do-usuario",
  "codigo": "FIN-ABC123",
  "email": "usuario@email.com",
  "roles": ["USER"],
  "iat": 1710000000,
  "exp": 1710001800,
  "jti": "uuid-unico-do-token" // JWT ID — permite blacklist por token individual
}
```

O campo `jti` (JWT ID) é essencial: ao fazer logout, você salva esse ID no Redis com TTL até a expiração original do token. Todo request passa por um filtro que verifica se o `jti` está na blacklist antes de autorizar — isso resolve o problema clássico de JWT não ser revogável.

No Spring Security, o fluxo fica:

```
Request → JwtAuthFilter → verifica assinatura → verifica expiração
                                     → verifica jti no Redis (blacklist) → extrai UserDetails → SecurityContext
```

Para os endpoints de automação (recebimento do PDF via N8N/WhatsApp), crie um mecanismo de **API Key** separado do JWT de usuário — um token estático por usuário, gerado no cadastro, enviado como header `X-API-Key`. Isso evita ter que negociar JWT com o bot do Telegram/WhatsApp, que tem fluxo diferente.

---

Quer que eu detalhe o fluxo completo de processamento do PDF (do recebimento no N8N até a tela de revisão), ou prefere que eu monte o ERD completo do banco de dados?